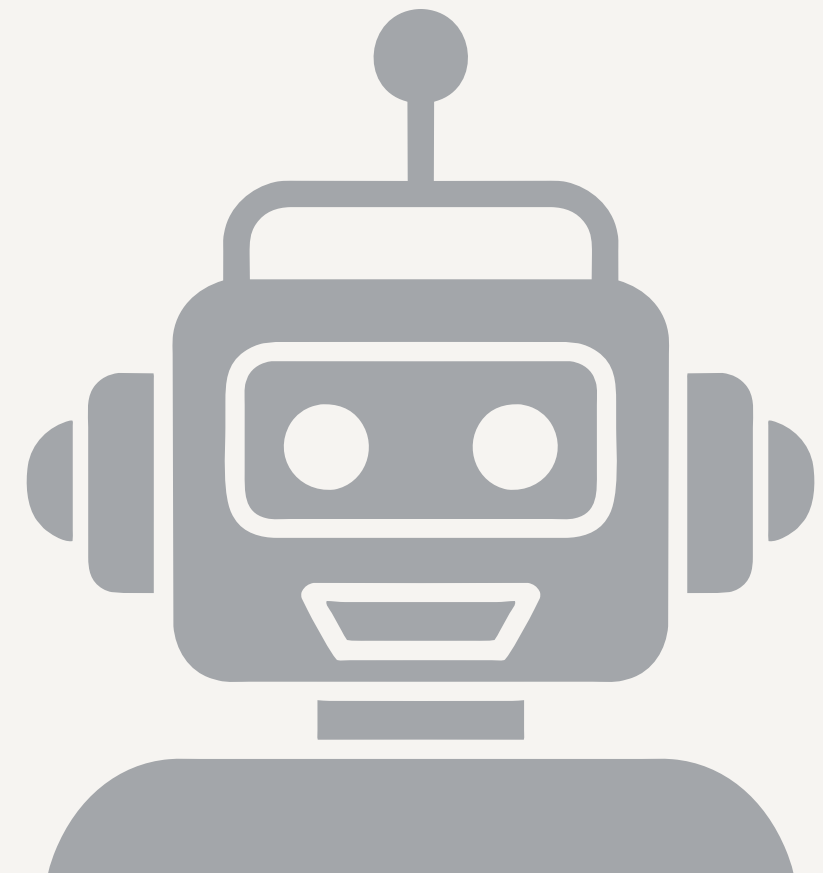


AI-POWERED CHAT SYSTEMS USING GROQ API

Presentation By:

Fathimath Riya Afreen M A



PROJECT OVERVIEW

Project 1: **AI Chatbot using Groq API**

- Built a conversational AI chatbot using Groq LLMs
- Supports real-time, multi-turn interactions with users.

Project 2: **RAG AI Chatbot**

- Built a RAG chatbot for PDF-based question answering
 - Retrieves relevant information from uploaded documents before generating responses.
-

TASK 1 :

AI CHATBOT

Developed an AI-powered chatbot using the Groq API that interacts with users in real time and generates intelligent responses.

Functions :

- Enables real-time conversations between users and AI.
 - Generates intelligent responses to user queries.
 - Supports customizable communication styles .
-

UNDERSTAND THE TERMS

- Semantic Search → Searches based on meaning, not just keywords
- Embeddings → Converts text into a format the AI can understand and compare
- Vector Database (ChromaDB) → Stores document information for quick searching
- RAG → A system that searches documents before generating answers

BUSINESS PURPOSE

Enhance user engagement through AI assistance.

Reduce manual effort in customer interactions.

Automate responses to common user queries.

KEY FEATURES

Real-time AI-generated responses

Multi-turn conversation memory

Customizable chatbot behaviour

Adjustable response creativity

CONTRIBUTIONS

07

- Set up project environment and dependencies
- Integrated Groq API with Python application
- Developed chatbot conversation logic
- Implemented message history management
- Tested different LLM models and parameters

CHALLENGES & SOLUTIONS

- Issue with deprecated model ; solved by replacing it with llama-3.1-8b-instant
- Response behaviour was inconsistent initially; improved it using prompt engineering and system prompts

TASK 2 :

RAG AI CHATBOT

- Built an AI assistant that can answer questions from uploaded PDF documents.
- The system identifies the most relevant information from the uploaded documents and uses AI to generate accurate answers.
- Implemented using a RAG (Retrieval-Augmented Generation) approach, where document content is retrieved first and then used by the AI to create responses.

BUSINESS PURPOSE & FEATURES

10

Business Purpose

- Reduce the time required to search through large documents.
- Provide quick and accurate answers from uploaded PDFs.
- Improve access to information and knowledge within documents.
- Enhance productivity by minimizing manual document review.

Key Features

- single or multiple PDF documents are supported.
- Semantic search.
- Context-aware AI-generated responses.
- Chat history for maintaining conversation flow.
- Source chunk display .
- User-friendly Streamlit interface.

Technologies used :

Technology	Purpose
Python	Main programming language used to build the system
Groq API	Provides the AI model that generates responses
Streamlit	Creates the chatbot's web interface
LangChain	Connects document retrieval with the AI model
ChromaDB	Stores document information for quick searching
HuggingFace Embeddings	Helps the system understand and compare document content

CONTRIBUTIONS

- Developed the RAG pipeline
- Implemented PDF processing and document retrieval.
- Integrated Groq API for AI-generated responses.
- Built the Streamlit user interface.
- Added chat history and source reference features.
- Optimized retrieval and answer accuracy.
- Implemented persistent ChromaDB storage

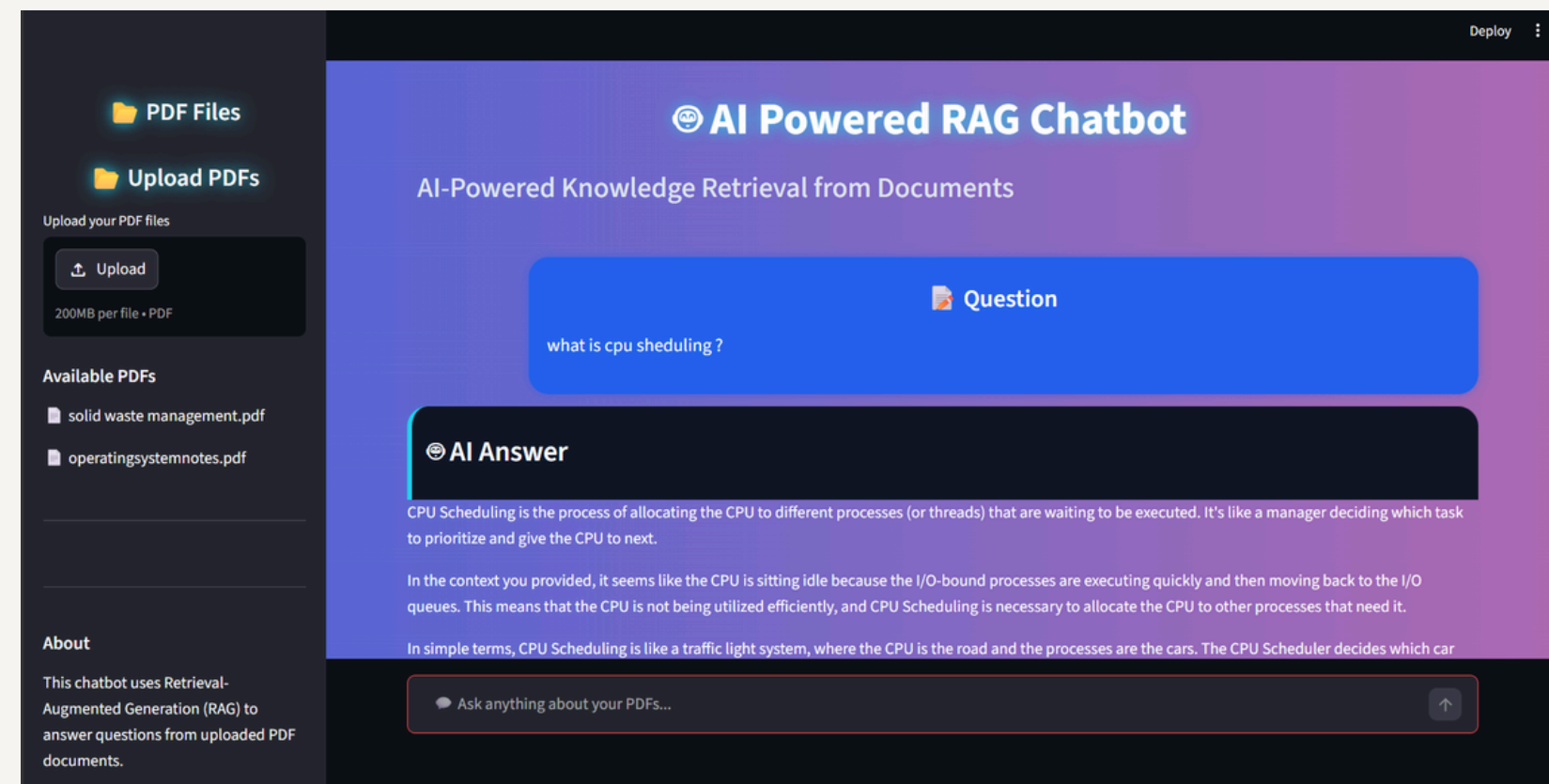
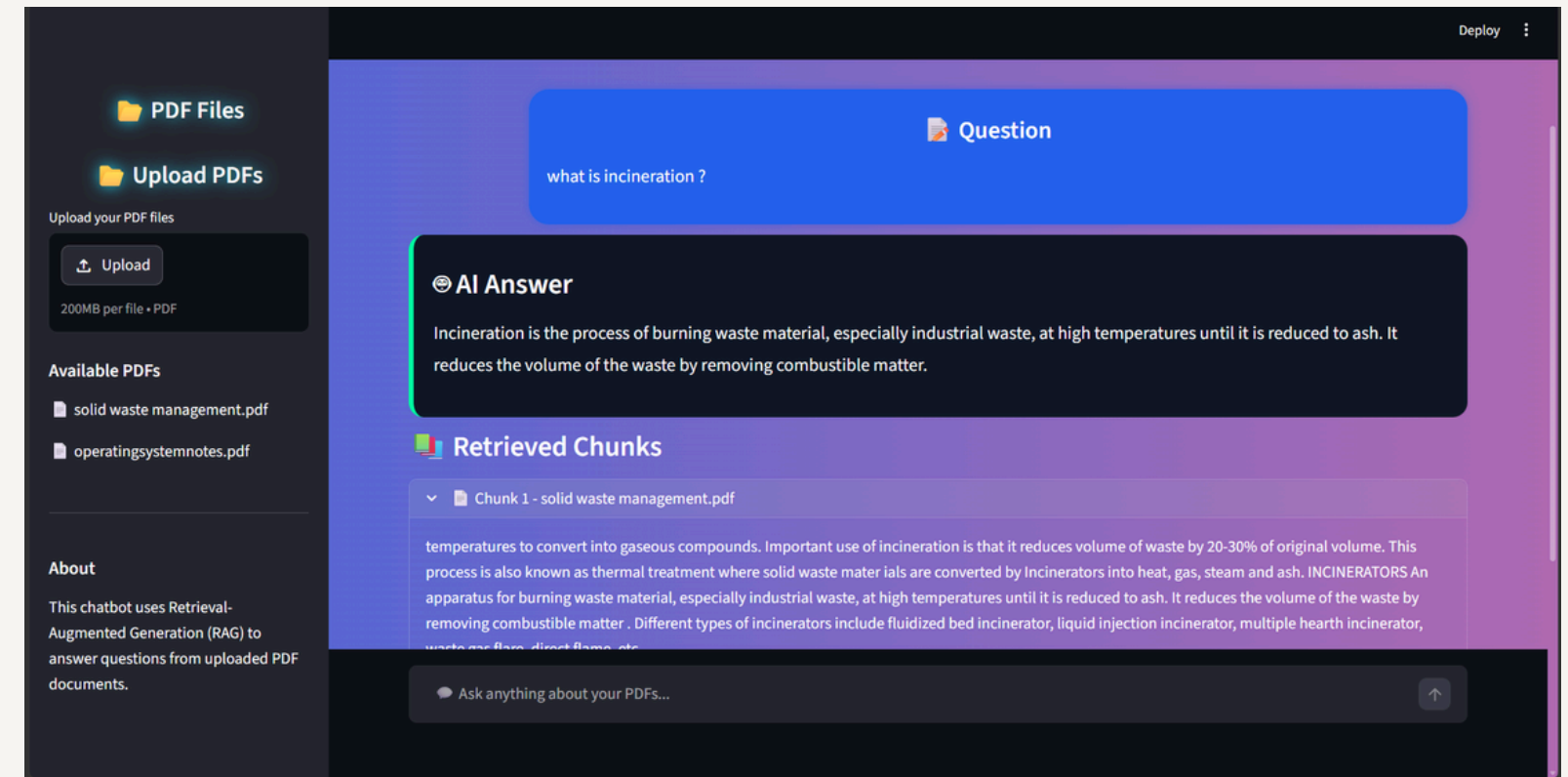
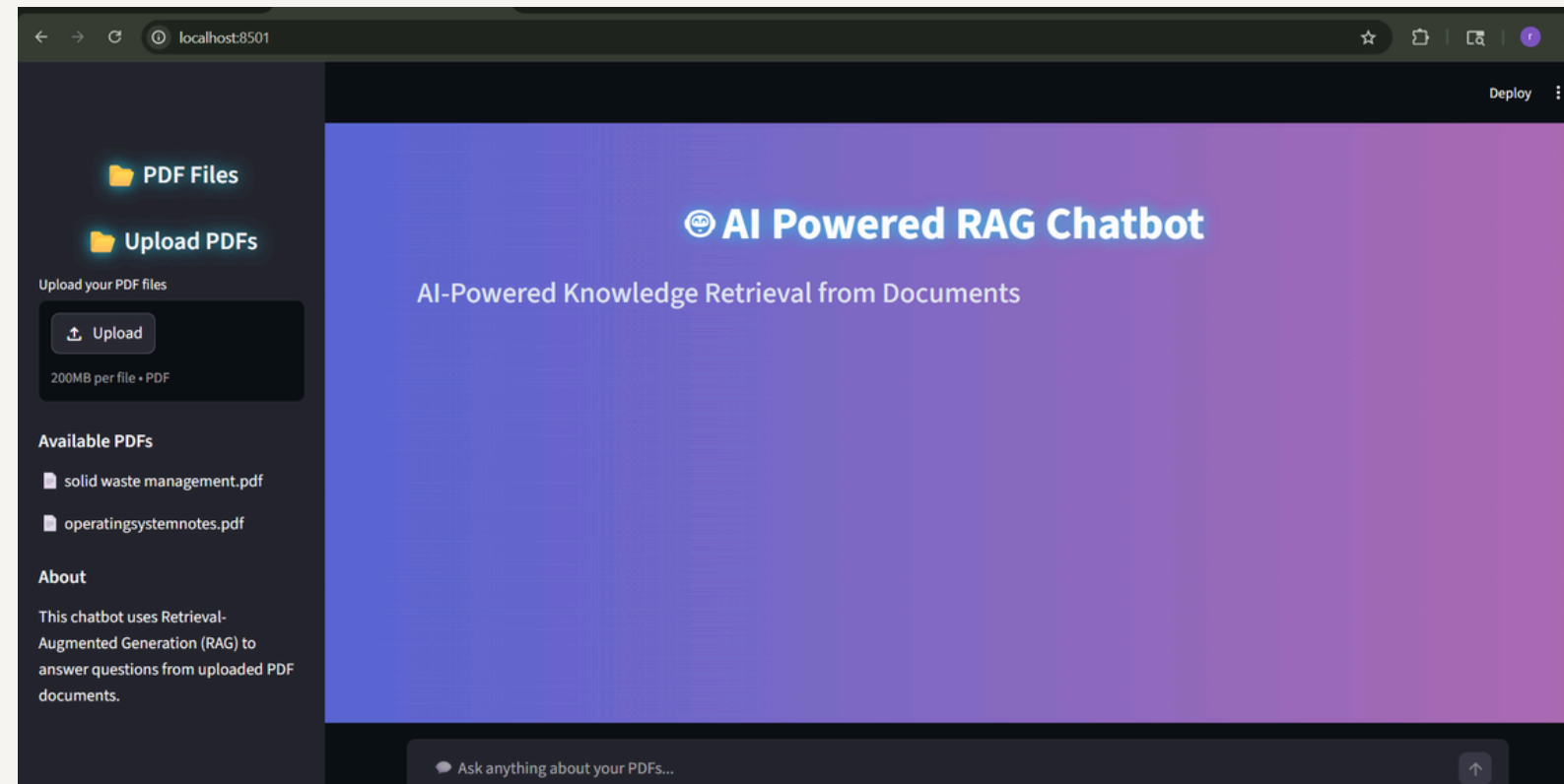
CHALLENGES & SOLUTIONS

- Implemented persistent ChromaDB storage to retain document data across sessions.
- Fixed issues with newly uploaded PDFs not being indexed correctly.
- Reduced inaccurate responses by improving retrieval and prompt design.

PROJECT OUTCOME

- Successfully built and tested a Retrieval-Augmented Generation (RAG) AI system.
- Implemented document-based question answering using PDF content instead of relying solely on LLM knowledge.
- Integrated semantic search, embeddings, and ChromaDB for efficient information retrieval.
- Added advanced features including multiple PDF support, chat history memory, source chunk display, and persistent database storage.
- Evaluated and improved system performance by experimenting with chunk size, retrieval settings, and prompt design.

SCREENSHOTS



CONCLUSION

Successfully developed and enhanced a Retrieval-Augmented Generation (RAG) AI system that enables users to upload PDF documents and receive accurate, context-aware answers through a chatbot interface.

The system was built using Python, Groq API, LangChain, ChromaDB, HuggingFace Embeddings, and Streamlit, combining document retrieval with AI-powered response generation. By implementing features such as semantic search, multiple PDF support, chat history, source chunk display, and persistent storage, the project demonstrated how Generative AI can be used to simplify information retrieval and improve access to knowledge from large documents.



THANK
YOU